Course & Quiz Platform (Mini LMS)

GROUP 5 | KalyanRam, Dileep, Jagan, Vineeth | Team size: 4 | Duration: 7 days

Backend	Frontend	Database	Auth	Deployment
Flask (REST API)	React + Vite	MongoDB + PyMongo	JWT (access + refresh)	Docker Compose

1. Problem Statement

Instructors author courses made of modules and lessons, attach auto-graded quizzes, and watch student progress; students enrol, learn, attempt timed quizzes and earn a certificate at 100% progress.

2. Roles & Permissions

Role	What they can do
Admin	Approves instructors, publishes/unpublishes courses, sees platform stats.
Instructor	Authors courses, modules, lessons and quizzes; views analytics.
Student	Enrols, consumes lessons, attempts quizzes, tracks progress.

3. Data Model

- **users** - credentials, role, profile.
- **courses** - title, description, level, instructor_id, status, and an embedded ordered array of modules, each with an ordered array of lessons (type: video URL / text, duration).
- **quizzes** - course_id, lesson_id, time limit, pass mark, negative marking, and an embedded array of questions (type, options, correct answers, marks).
- **enrollments** - student_id, course_id, enrolled_at, status.
- **attempts** - quiz_id, student_id, started_at, expires_at, submitted_at, answers, score, per-question result.
- **progress** - student_id, course_id, completed lesson ids, percent, certificate_issued.

4. Functional Requirements

- Course authoring: add, reorder and remove modules and lessons; draft then publish.
- Quiz builder supporting MCQ, multi-select and true/false with marks and optional negative marking.
- Enrolment, lesson completion tracking and resume-where-you-left-off.
- Timed quiz attempt with a server-recorded deadline, auto-grading on submit.
- Attempt review showing the correct answer and explanation per question.
- Instructor analytics: average score, per-question accuracy, drop-off by lesson, enrolment count.
- Certificate record created automatically when progress reaches 100%.

5. Core Technical Challenge

Correct answers never leave the server before submission. The start-attempt endpoint returns questions with the answer keys stripped and stores `started_at` + `expires_at` in the attempt document. Grading happens entirely server-side against the stored key. A submission arriving after `expires_at` is graded on whatever was saved, and a stale attempt is auto-submitted. One in-progress attempt per quiz per student; attempt limits enforced. Reordering modules must not break existing progress records.

6. React Screens

- Course catalogue with level and search filters.
- Course player: lesson content, sidebar with progress ticks, next/previous.
- Quiz runner with countdown timer and per-question navigation.
- Result screen and attempt review.
- Instructor authoring UI with drag-to-reorder modules and lessons.
- Analytics dashboard; admin console.

7. Authentication & Authorisation Specification

- Passwords hashed with bcrypt - plain text passwords must never touch the database.
- Endpoints: `POST /auth/register`, `/auth/login`, `/auth/refresh`, `/auth/logout`, `GET /auth/me`.
- The access token carries a `role` claim; a custom decorator `@role_required("admin")` guards privileged routes on top of `@jwt_required()`.
- Refresh rotation: a refresh call issues a new access token; logout adds the token JTI to a blocklist so it cannot be reused.
- React side: Axios request interceptor attaches the token, response interceptor catches 401, calls refresh once, retries the original request, and logs the user out if refresh also fails.
- Route protection in React by role - a seeker must never be able to open an admin screen, even by typing the URL.

8. Docker Specification

- `backend/Dockerfile` - Python slim base, requirements installed first for layer caching, served by `gunicorn` (not the Flask dev server).
- `frontend/Dockerfile` - multi-stage: Node stage runs `npm run build`, nginx stage serves the static build and proxies `/api` to the backend.
- `docker-compose.yml` - services `api`, `web`, `db` on one network, a named volume for database data, a healthcheck on `db`, and `depends_on` so the API waits for it.
- All secrets (DB URL, JWT secret key) come from `.env`; `.env.example` is committed, `.env` is not.
- Acceptance test: on a clean machine, `git clone` then `docker compose up --build` gives a working app with seeded demo data.

9. Phase-wise Execution Plan

Phase 1 - Day 1: Foundation

- Finalise the data model and draw the ER diagram / collection diagram before writing code.
- Repo and folder structure: `backend/`, `frontend/`, `docker-compose.yml`, `README.md`.
- Docker skeleton up early - database container running and reachable from the API container on day one.
- User model + full JWT auth flow + role seeding. Do not move on until login and refresh work in Postman.

Phase 2 - Day 2 to 3: Core API

- Every core CRUD endpoint with request validation and clear error responses (400 / 401 / 403 / 404 / 409).
- Role guards on every write route; ownership checks so a user can only touch their own records.
- Pagination, filtering and sorting on all list endpoints - never return the whole table.
- Test every endpoint in Postman and save the collection before you write a single React component.

Phase 3 - Day 3 to 5: React Application

- Routing, auth context, protected route wrapper, login and register screens.
- List, detail and create/edit screens for the main entities with client-side form validation.
- Loading, empty and error states on every screen - no blank page while data is fetching.
- Shared Axios instance; no API URL hardcoded in components.

Phase 4 - Day 5 to 6: Core Business Logic

- Implement the core rule described above - this is what separates this project from a CRUD demo.
- Dashboard and aggregation endpoints computed in the database layer, not in JavaScript.
- Search and filters wired end to end.

Phase 5 - Day 6 to 8: Harden and Ship

- Full dockerization verified from a clean clone; fix anything that only worked on your laptop.
- Seed script that creates demo users for each role plus realistic sample data.
- README: architecture overview, setup steps, environment variables, API endpoint table, demo credentials.
- Postman collection, bug fixing pass, and a 5-minute demo recording walking through each role.

10. Work Split

Owner	Responsibility
Member 1	Auth, roles, users, admin console, enrolment API.
Member 2	Course authoring API - nested modules and lessons, ordering, publish workflow.
Member 3	Quiz engine: builder API, attempt lifecycle, timer, grading, analytics aggregations.
Member 4	React application: catalogue, player, quiz runner, authoring UI, dashboards.

Every member must commit backend code and review at least one pull request - no single member owns the whole repository.

11. Deliverables

- GitHub repository containing both applications with a readable commit history from every member.
- README.md with setup instructions, environment variables and an API endpoint table.
- ER diagram (SQL) or collection diagram (MongoDB).
- Seed script with demo data and demo credentials per role.
- Postman collection covering every endpoint.
- Working `docker compose up --build` on a clean machine.
- 5-minute demo video.

12. Evaluation

Area	Weight	What is checked
Functionality	30%	All listed features work end to end without manual database edits
API design & JWT	20%	Correct status codes, validation, role guards, refresh and blocklist working
Core business logic	20%	The project-specific core rule is correctly and safely implemented
React frontend	15%	Component structure, state handling, validation, loading and error states

Area	Weight	What is checked
Docker & docs	15%	One-command startup, clean compose file, complete README and seed data

13. Stretch Goals (only after everything above works)

- Question bank with random question selection per attempt.
- Downloadable certificate.